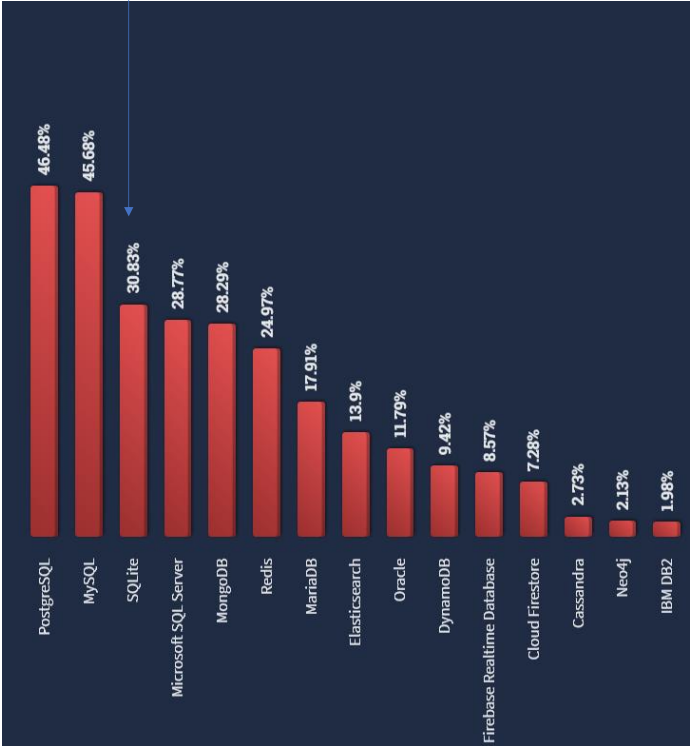


Linguagem SQL

Nesta disciplina iremos utilizar como SGBD o SQLite

<https://sqliteonline.com>

Bancos de Dados mais populares em 2023



3º SGBD mais popular no mundo

O SQLite reconhece apenas cinco tipos:

Tipo	Descrição
INTEGER	Números inteiros (qualquer tamanho)
REAL	Números de ponto flutuante (float, double)
TEXT	Sequências de caracteres (strings)
BLOB	Dados binários (Binary Large Object)
NULL	Valor nulo (ausência de dado)

o número do Id vai ser automaticamente inserido
toda vez que se cria um novo registro

```
1 CREATE TABLE IF NOT EXISTS Alunos (  
2     Id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     Nome TEXT  
4 );  
5  
6 INSERT INTO Alunos(Nome) VALUES("Joao");  
7 INSERT INTO Alunos(Nome) VALUES("Maria");  
8  
9 SELECT * FROM Alunos;
```

Vc precisa especificar
quais colunas você está
adicionando o valor

```
1 CREATE TABLE IF NOT EXISTS Alunos (
```

```
2   Id INTEGER,
```

```
3   Nome TEXT,
```

```
4   PRIMARY KEY(Id)
```

```
5 );
```

```
6
```

```
7 INSERT INTO Alunos(Nome) VALUES("Joao");
```

```
8 INSERT INTO Alunos(Nome) VALUES("Maria");
```

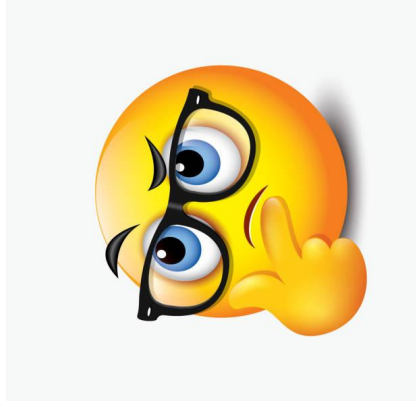
```
9
```

```
10 SELECT * FROM Alunos;
```

Você pode declarar a chave primária assim também, porém não vai poder utilizar o AUTOINCREMENT

Eu posso ter uma tabela
sem chave primária?

Sim! Mas...



O que são CONSTRAINTS?

- São regras aplicadas ao banco de dados que permitem que você configure as **características** de uma determinada coluna:

Por exemplo:

- 1 - Se uma coluna aceita ou não valores nulos
- 2 - Se uma coluna é ou não uma chave primária
- E assim vai...

Por que usar CONSTRAINTS?

- A princípio, quando criamos as tabelas do banco de dados, elas aceitam qualquer valor.
- Por exemplo, se eu tenho uma coluna para armazenar a informação da placa de um carro, nada me impede de simplesmente não cadastrar essa informação. Mas, será que isso faz sentido?
- Ou uma coluna que armazena a sigla do estado de uma loja, faria sentido alguém cadastrar o texto "Distrito Federal"?
- Então utilizamos CONSTRAINTS para deixar o banco de dados menos sujeito a erros e mais confiável e robusto

Quais tipos de CONSTRAINTS existem?

- Existem vários tipos de CONSTRAINTS e algumas das principais são:

NOT NULL

DEFAULT

UNIQUE

CHECK

PRIMARY KEY

FOREIGN KEY

Quais tipos de CONSTRAINTS existem?

- Existem vários tipos de CONSTRAINTS e algumas das principais são:

NOT NULL

DEFAULT

UNIQUE

CHECK

PRIMARY KEY

FOREIGN KEY

```
1 CREATE TABLE IF NOT EXISTS Alunos (  
2   Id INTEGER,  
3   Nome TEXT NOT NULL,  
4   PRIMARY KEY(Id)  
5 );
```



Quais tipos de CONSTRAINTS existem?

- Existem vários tipos de CONSTRAINTS e algumas das principais são:

NOT NULL

DEFAULT

UNIQUE

CHECK

PRIMARY KEY

FOREIGN KEY

```
1 CREATE TABLE IF NOT EXISTS Alunos (  
2   Id INTEGER,  
3   Nome TEXT DEFAULT "zezinho",  
4   PRIMARY KEY (Id)  
5 );  
6  
7 INSERT INTO Alunos (Id) VALUES (1);  
8 INSERT INTO Alunos (Nome) VALUES ("Maria");
```

Esse comando será executado?

Quais tipos de CONSTRAINTS existem?

- Existem vários tipos de CONSTRAINTS e algumas das principais são:

NOT NULL

DEFAULT

UNIQUE

CHECK

PRIMARY KEY

FOREIGN KEY

```
1 CREATE TABLE IF NOT EXISTS Alunos (  
2   Id INTEGER,  
3   Nome TEXT UNIQUE,  
4   PRIMARY KEY(Id)  
5 );  
6  
7 INSERT INTO Alunos(Id, Nome) VALUES(1, "João");  
8 INSERT INTO Alunos(Id, Nome) VALUES(2, "João");
```

Quais tipos de CONSTRAINTS existem?

- Existem vários tipos de CONSTRAINTS e algumas das principais são:

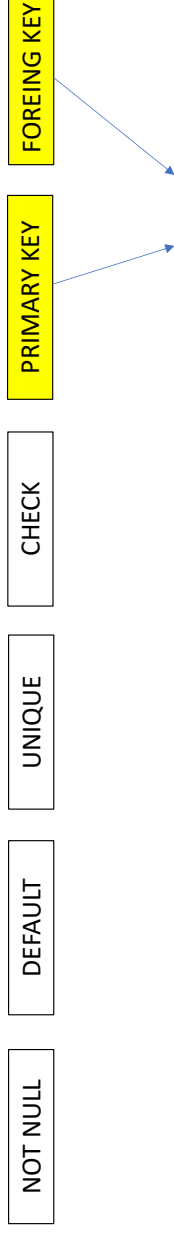
NOT NULL	DEFAULT	UNIQUE	CHECK	PRIMARY KEY	FOREIGN KEY
----------	---------	--------	-------	-------------	-------------

```
1 CREATE TABLE IF NOT EXISTS Alunos (  
2   Id INTEGER,  
3   Nome TEXT CHECK(length("Nome") >= 4),  
4   PRIMARY KEY(Id)  
5 );  
6  
7 INSERT INTO Alunos(Id, Nome) VALUES(1, "João");  
8 INSERT INTO Alunos(Id, Nome) VALUES(2, "Maria");  
9 INSERT INTO Alunos(Id, Nome) VALUES(3, "Isa");
```

Não vou
conseguir
adicionar

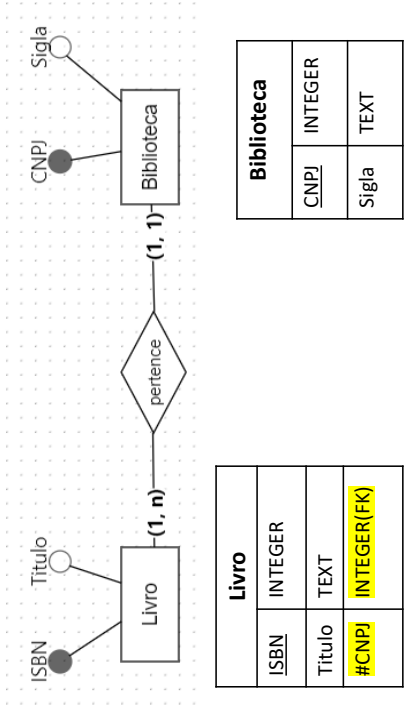
Quais tipos de CONSTRAINTS existem?

- Existem vários tipos de CONSTRAINTS e algumas das principais são:



Vou mostrar a utilização dessas duas constraints no próximo exemplo

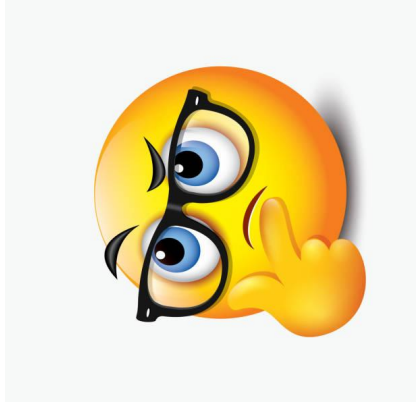
FOREING KEY



```
1 CREATE TABLE Livro (
2   ISBN INTEGER,
3   Titulo TEXT NOT NULL,
4   CNPJ INTEGER,
5   PRIMARY KEY(ISBN),
6   FOREIGN KEY(CNPJ) REFERENCES Biblioteca(CNPJ)
7 );
```

```
1 CREATE TABLE Biblioteca (
2   CNPJ INTEGER,
3   Sigla TEXT NOT NULL,
4   PRIMARY KEY(CNPJ)
5 );
```

No exemplo anterior, se a biblioteca deixar de existir, o que vai acontecer a chave estrangeira?



E se o CNPJ da biblioteca mudar de número, o que vai acontecer a chave estrangeira?

Ações ON DELETE e ON UPDATE

- Temos 5 opções para tratar remoção e/ou atualização de chaves estrangeiras:

NO ACTION	Não faz nada, deixa como está. Essa é a opção tradicional
SET NULL	Coloca NULL no valor
SET DEFAULT	Coloca um valor DEFAULT no valor
CASCADE	Propaga a remoção e/ou atualização para todas as tabelas que fazem referência à chave primária da tabela
RESTRICT	Não permite a remoção e/ou atualização do dado

Habilitar o uso de chaves estrangeiras
(necessário no SQLite)

```
1 PRAGMA foreign_keys = ON;
2
3 CREATE TABLE Biblioteca (
4     CNPJ INTEGER,
5     Sigla TEXT NOT NULL,
6     PRIMARY KEY(CNPJ)
7 );
8
9 CREATE TABLE Livro (
10     ISBN INTEGER,
11     Titulo TEXT NOT NULL,
12     CNPJ INTEGER,
13     PRIMARY KEY(ISBN),
14     FOREIGN KEY(CNPJ) REFERENCES Biblioteca(CNPJ)
15         ON DELETE CASCADE
16         ON UPDATE CASCADE
17 );
18
19 INSERT INTO Biblioteca VALUES (123, "fga");
20 INSERT INTO Livro VALUES (555, "titulo", 123);
21 UPDATE Biblioteca SET CNPJ = 777 WHERE CNPJ = 123;
```

Inserir dados na tabela

```
CREATE TABLE Aluno (  
  Matricula INTEGER,  
  Nome TEXT,  
  Idade INTEGER,  
  PRIMARY KEY(Matricula)  
);
```

Opção n° 1 (tradicional)

```
INSERT INTO Aluno(Matricula, Nome, Idade) VALUES (123, "João", 22);
```

Opção n° 2

Se eu for adicionar todos os valores, não preciso especificar as colunas

```
INSERT INTO Aluno VALUES (123, "João", 22);
```

Adicionando múltiplos valores ao mesmo tempo

```
INSERT INTO Aluno VALUES  
(111, "João", 22),  
(222, "Maria", 22),  
(333, "José", 22);
```

Remover dados da tabela

```
CREATE TABLE Aluno (  
  Matricula INTEGER,  
  Nome TEXT,  
  Idade INTEGER,  
  PRIMARY KEY(Matricula)  
);
```

```
INSERT INTO Aluno VALUES  
(111, "João", 22),  
(222, "Maria", 22),  
(333, "José", 22);
```

Opção nº 1 (remover todos os dados da tabela)

```
DELETE FROM Aluno;
```

Opção nº 2 (remover tupla específica)

```
DELETE FROM Aluno WHERE Nome = "Maria";
```

Atualizar dados na tabela

```
CREATE TABLE Aluno (  
  Matricula INTEGER,  
  Nome TEXT,  
  Idade INTEGER,  
  PRIMARY KEY(Matricula)  
);
```

```
INSERT INTO Aluno VALUES  
(111, "João", 22),  
(222, "Maria", 22),  
(333, "José", 22);
```

Atualizar o nome da Maria para Joana

```
UPDATE Aluno SET Nome = "Joana" WHERE Nome = "Maria";
```

Atualizar todos os nomes da tabela Aluno para Joana

```
UPDATE Aluno SET Nome = "Joana";
```

Alterar dados da tabela

O comando **ALTER TABLE** é utilizado para **modificar a estrutura de uma tabela já existente** em um banco de dados. Ele permite fazer alterações como:

- Adicionar colunas

```
1 ALTER TABLE nome_da_tabela  
2 ADD COLUMN nome_coluna tipo_dado;
```

- Renomear colunas

```
1 ALTER TABLE nome_da_tabela  
2 RENAME COLUMN nome_atual TO novo_nome;
```

- Renomear a própria tabela.

```
1 ALTER TABLE nome_atual  
2 RENAME TO novo_nome;
```